Database Design Report

English Premier League Database Application

Prepared For	Course	Prepared By	Version	Date
COSC 457 Database Management Systems	Towson University	Abel Mbomnda / Project Team	1.0	2026-03-23

This report presents the proposed design for a business database application based on the English Premier League (EPL). The project models the operational side of league management and provides a practical foundation for entity-relationship modeling, relational schema development, SQL reporting, and Java Swing user-interface implementation.

1. Introduction

The English Premier League is a strong real-world business domain for a database project because it combines rich operational data with clear business rules. A league season includes registered clubs, player rosters, fixtures, match events, transfers, standings, and management reports. These activities naturally produce connected entities and meaningful relationships that can be translated into a robust ERD and a useful application.

The proposed system, called the EPL Database Management System, will help administrators capture and manage football business data in a structured way. Instead of relying on scattered spreadsheets or manual tracking, the database will centralize records and support both transaction processing and reporting. The final software will be implemented with Java Swing on the front end and MySQL on the back end using JDBC connectivity.

2. Target Business

2.1 Business Background

This project treats the target business as a professional football league administration environment. The league coordinates multiple clubs that compete during a single season structure built around matchweeks. Each club maintains its own registered roster, manager, stadium, and season performance. League administrators need reliable access to scheduling data, match outcomes, player movement, and standings in order to support operational decision-making and produce reports.

2.2 Business Profile

- Business type: professional football competition and league administration
- Operational scope: seasonal competition management for twenty clubs
- Primary records: clubs, players, managers, stadiums, seasons, fixtures, results, events, transfers, standings, and sponsors
- Primary users: system administrator, reporting analyst, and data entry staff
- Core value of the database: consistent data capture, efficient search, and faster reporting

2.3 Business Activities

- Create seasons and matchweeks.
- Register clubs and maintain club profiles.
- Register players and attach them to clubs for a season.
- Schedule fixtures and assign dates, times, and venues.
- Capture scores and detailed match events such as goals, cards, and substitutions.
- Maintain transfer history across windows.
- Generate standings, top scorer reports, and other management summaries.

3. Business Process

The business process begins before the season starts, when the administrator creates the new season and records club, stadium, and manager information. Players are then registered to clubs for that season, which creates the official squad records. After registration, fixtures are scheduled by matchweek, including home club, away club, venue, and kickoff time. During and after each match, the administrator records the final score and important match events. Transfer windows allow player movement between clubs while preserving historical records.

At any point, reports can be run to see standings, player statistics, fixture schedules, and club performance trends.

Core workflow for the EPL database application.

3.1 Example Activity Scenario

1. Administrator creates season 2026/2027 and defines matchweek 1 through matchweek 38.
2. Administrator registers Arsenal FC, Liverpool FC, and other participating clubs.
3. Players are added to the system and then registered to clubs for the selected season.
4. A fixture is scheduled: Arsenal FC vs Liverpool FC at Emirates Stadium on 2026-08-17.
5. After the match, the result 2-1 is saved and the goal events are recorded.
6. Standings are refreshed and reports become available to management users.

4. User Requirements

4.1 User Groups

- Administrator: full maintenance access to master data, fixtures, results, and transfers.
- Reporting user: read-only access to league reports and search screens.
- Data entry clerk: controlled access to operational screens such as result entry and player registration.

4.2 Functional Requirements

ID	Requirement	User
FR-01	Create, update, and deactivate seasons	Administrator
FR-02	Create, update, and search clubs	Administrator
FR-03	Maintain stadium and manager details	Administrator
FR-04	Create player master records	Administrator / Clerk
FR-05	Register a player to a club for a season	Administrator / Clerk
FR-06	Schedule fixtures by season and matchweek	Administrator
FR-07	Record final scores and match events	Administrator / Clerk
FR-08	Update league standings from match results	System / Administrator
FR-09	Process player transfers between clubs	Administrator
FR-10	Search fixtures by club, date, or matchweek	All users
FR-11	Produce standings and top scorer reports	Administrator / Reporting user
FR-12	Display roster and transfer	Administrator / Reporting user

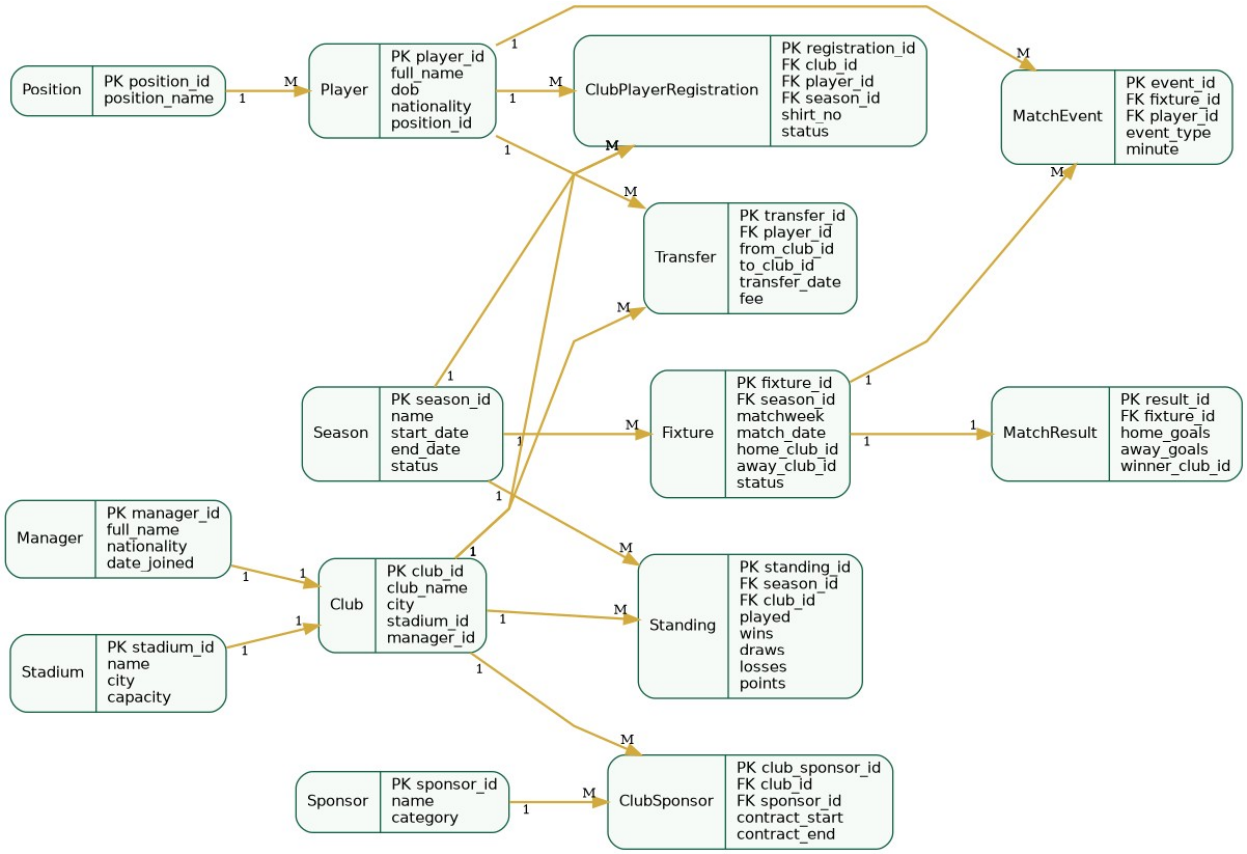
	history for a club	

4.3 Nonfunctional Requirements

- The application must be implemented in Java Swing and should run without additional configuration beyond Java and database access.
- All database updates should use prepared statements and input validation.
- The interface should be easy to navigate and visually organized by module.
- The application should return search and report results in a readable table format.
- Sample data should be realistic enough to demonstrate joins, aggregations, filters, and ordering queries.

5. Data Model

The data model is centered on the season, club, player, fixture, result, and standing entities. Supporting entities such as manager, stadium, position, sponsor, and transfer history strengthen the realism of the model and provide richer SQL reporting opportunities.



Conceptual ERD for the proposed system.

5.1 Main Entities and Purpose

Entity	Purpose

Season	Stores each competition season and its active state.
Club	Stores club identity and operating information.
Player	Stores player master data independent of club movement.
ClubPlayerRegistration	Tracks which player belongs to which club for a season.
Fixture	Stores scheduled matches including home club and away club.
MatchResult	Stores final score for each fixture.
MatchEvent	Stores goals, cards, substitutions, and time-based events.
Standing	Stores club performance totals for a season.
Transfer	Stores player movement between clubs over time.
Sponsor / ClubSponsor	Supports optional commercial relationship tracking.

5.2 Key Business Rules

- A fixture belongs to exactly one season and one matchweek.
- Each fixture must involve two different clubs.
- Each match result belongs to one fixture.
- A player can have many registration records over time, but only one active club registration for a given season period.
- A standing record is unique per club per season.
- Transfer history must be preserved even after a player changes clubs.

5.3 Relational Schema Summary

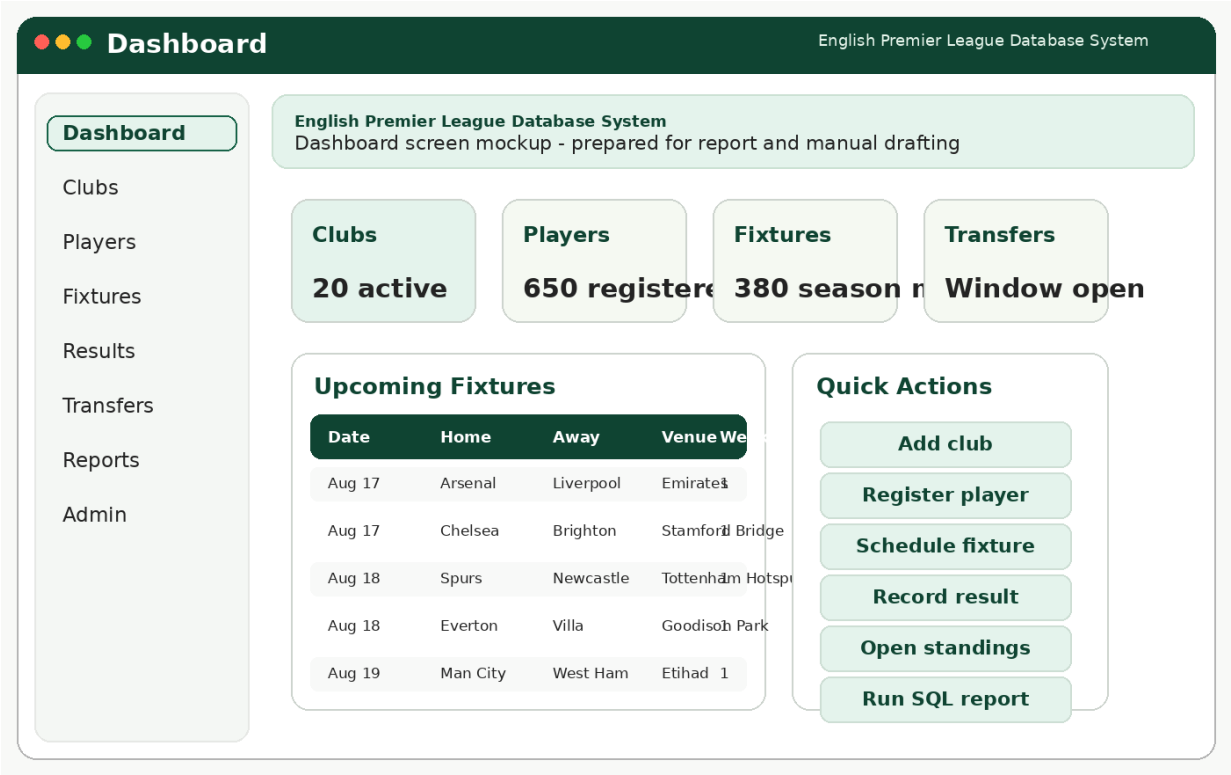
```
Season(season_id PK, name, start_date, end_date, status)

Stadium(stadium_id PK, name, city, capacity)

Manager(manager_id PK, full_name, nationality, date_joined)

Club(club_id PK, club_name, city, stadium_id FK, manager_id FK)

Position(position_id PK, position_name)

Player(player_id PK, full_name, dob, nationality, position_id FK)

ClubPlayerRegistration(registration_id PK, club_id FK, player_id FK, season_id FK,
shirt_no, status)

Fixture(fixture_id PK, season_id FK, matchweek, match_date, home_club_id FK,
away_club_id FK, status)

MatchResult(result_id PK, fixture_id FK, home_goals, away_goals, winner_club_id FK)

MatchEvent(event_id PK, fixture_id FK, player_id FK, event_type, minute)

Transfer(transfer_id PK, player_id FK, from_club_id FK, to_club_id FK, transfer_date,
fee)
```

```
Standing(standing_id PK, season_id FK, club_id FK, played, wins, draws, losses,
goals_for, goals_against, goal_difference, points)
```

6. Expected SQL Reports

Report	Business Value
List all players in a selected club roster	supports squad review and roster verification
Show fixtures for a selected matchweek	supports weekly operations and planning
Display league standings for a selected season	supports performance monitoring
Find top scorers in the season	supports media and analytics reporting
Show transfer history for a club or player	supports player movement analysis
Show clubs with the most wins	supports performance comparison
List fixtures played in a selected stadium	supports venue usage review
Find players with the most yellow or red cards	supports discipline analysis

7. Application Design Notes



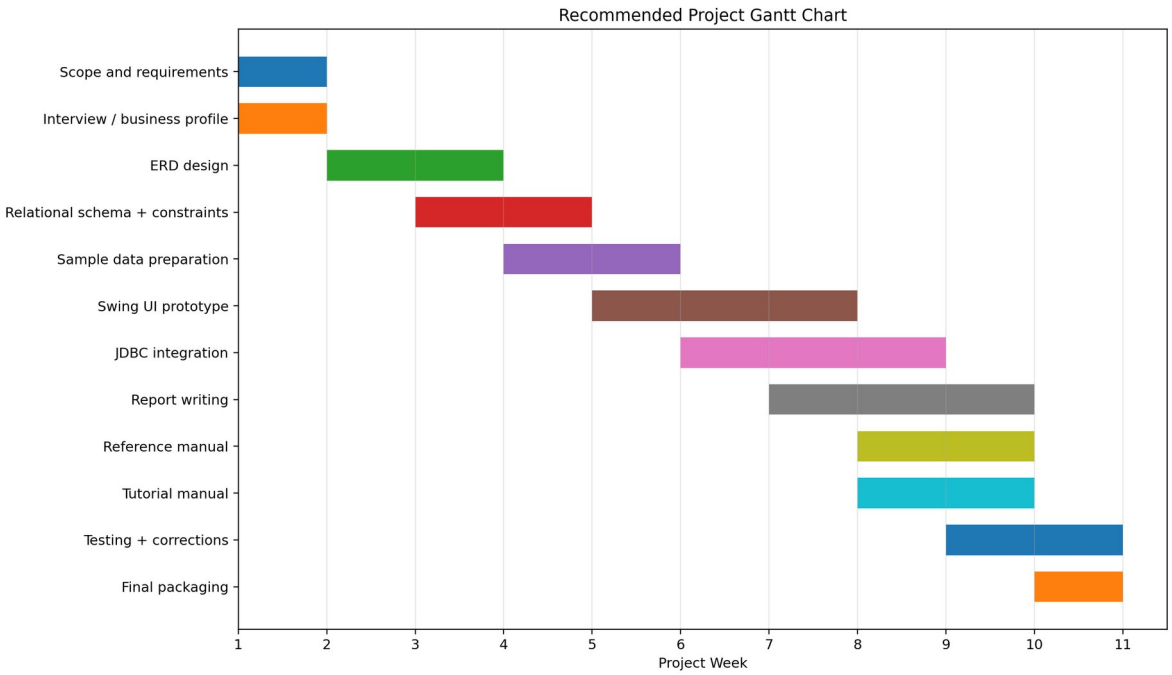
Prototype dashboard screen used to guide final Swing implementation.

The application will use a menu-driven desktop layout. Each major module opens a form for data entry or a grid view for search and reporting. Prototype screens are included in this package to accelerate documentation and interface planning; they should be replaced with live screenshots once the application is implemented.

8. Assumptions and Limitations

- The model assumes one league competition rather than multiple domestic or continental competitions.
- Financial operations such as payroll and ticketing are outside the minimum scope.
- The sponsor module is optional and may be simplified if time becomes limited.
- The system is intended for academic use and therefore does not include enterprise authentication or online public access.

9. Project Plan



Recommended work breakdown and timeline.

10. Conclusion

The EPL database application is a strong and realistic semester project because it combines rich operational data, clear business rules, and meaningful reporting requirements. It is complex enough to support a solid ERD and nontrivial SQL queries, yet structured enough to implement successfully with Java Swing and MySQL. With disciplined scope control, realistic sample data, and polished documentation, the project can become a complete and convincing database business application.